

Final Project - LLM

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

Project scope

Project Scope

Build a working LLM-powered application on a knowledge base of your choice. In most projects, this means a RAG application. You will apply modules 1 to 5 to your own data to build, evaluate, and document an end-to-end system.

Past Cohort Examples

Previous projects give a great sense of the expected scope. Common examples include a recipe search assistant, a fitness assistant, or a music notation reference tool. Most are modest in scope but cleanly built and well-documented.

Your project should:

- **Pick a domain:** Choose a specific topic or field.
- **Ingest & Index:** Load your knowledge base and index it for retrieval.
- **Wire up LLM:** Connect the retrieval system to a language model.
- **Evaluate:** Test retrieval performance and the end-to-end RAG flow.
- **Monitoring:** Ideally include observability and monitoring.

Choosing a project idea

A good project starts with a **real question someone would ask**, not with a tool choice. Before deciding on your stack, write down the fundamental parameters of your application.

Your project should define:

- **The target user:** Who is this assistant actually built for?
- **The user need:** What specific task do they need help with?
- **The knowledge base:** What exact data source will the assistant query?
- **The ideal output:** What does a high-quality, useful answer look like?
- **Success metrics:** How will you evaluate if the answer is accurate?

Good ideas usually have these properties:

- **Specific Domain:** The topic is niche enough that standard, generic ChatGPT fails to answer well.
- **Accessible Data:** The knowledge base is clean, accessible, and large enough for meaningful retrieval.
- **Verifiability:** Answers can be explicitly cross-referenced and checked against the source documents.
- **Focused Scope:** The entire prototype can be fully built and evaluated within a few weeks.
- **Clear Readme:** The project documentation easily explains the core domain problem to outsiders.

Choosing a project idea

Avoid vague ideas like “**a chatbot for documents**” unless you make the user, documents, and task concrete. For example:

01 • Document Chatbot

Instead of: Generic PDF chatbot

Build: An assistant for a specific set of product manuals, public policies, internal notes, or course materials.

02 • Medical Assistant

Instead of: "Medical assistant"

Build: A tool that answers questions from a public clinical guideline dataset and clearly cites sources.

03 • Career Assistant

Instead of: "Career assistant"

Build: A tool that searches a curated set of job descriptions and explains which skills are repeatedly requested.

Using the AI Engineering Guide

The AI Engineering Guide is an excellent source of project ideas. It collects real-world take-home assignments from [100+ company GitHub repositories](#).

Do not copy blindly. Use it as a prompt to:

- **Pick the right work:** Choose an assignment that aligns with a real AI engineering role you want.
- **Identify the core problem:** Find and focus on the fundamental user need.
- **Use public data:** Replace missing or proprietary datasets with shareable, public data.
- **Keep scope realistic:** Ensure it is small enough to finish, but robust enough for your portfolio.

Leverage Your Job Search

The guide is especially valuable if you want your project to support your job search.

How to execute:

Look at assignments from companies or roles you care about, then build a smaller version that demonstrates those exact skills.

Knowledge base / dataset

You choose the knowledge base. It can be in any language, including non-English. The README must still be in English so peer reviewers can evaluate the project. Pick a domain you find interesting because you will work with it for several weeks.

Avoid

- **Course datasets:** Any dataset the course uses in its lectures or homework. Choose something new so the project shows your own work.
- **Toy data:** Trivially small or toy data that does not exercise retrieval meaningfully.

Strong Sources

- **Open-source docs:** Documentation for an open-source project.
- **Official files:** Public policy, legal, or government documents.
- **Technical manuals:** Product manuals or technical support pages.
- **Personal notes:** Your own notes, if you can make enough of them public for reviewers.

Required components

Every LLM project must include:

- **Ingestion path:** Turns your raw knowledge base into something searchable.
- **Retrieval system:** Text search, vector search, or hybrid.
- **LLM answering layer:** Uses the retrieved documents to answer.
- **Retrieval evaluation:** Which strategy was tried and how it scored.
- **End-to-end evaluation:** Assessment of the full RAG flow.
- **User interface:** Script, notebook, or web UI ([Streamlit](#), [Gradio](#), [Flask](#)).

Nice-to-Haves

The grading rubric does **not** require these components for completion:

- **Advanced Monitoring:** Grafana, PostgreSQL, or other production tracking tools.
- **Live Deployment:** A fully deployed live system on cloud infrastructure.

Tech stack flexibility

You are not restricted to the technologies covered in the course. Use any LLM provider, vector database, framework, or programming language.

Allowed Technologies

- **Any LLM provider:** Use any API provider, local models, or endpoints.
- **Vector databases:** Choose any vector store or hybrid index system.
- **Any framework:** Standard libraries, LangChain, LlamaIndex, etc.
- **Languages:** Python, JavaScript, Go, or other backend options.

Limitations & Requirements

- **Document in README:** Clearly explain choices. Peer reviewers might not know your stack.
- **Reproducibility:** If a reviewer cannot run it or understand how it runs, you lose points.
- **Non-Python setup:** Give explicit instructions for JS, Go, etc. Python is the only assumed stack.

Frameworks

Guidelines & Tools

- **No-Framework Learning:**
The course teaches RAG without frameworks so that you understand the underlying concepts.
- **Frameworks Allowed:**
For your final project, you are fully permitted to use LangChain, LlamaIndex, Haystack, or similar tools.

Documentation Requirements

- **Explain Your Choice:**
Clearly document why you chose your specific framework over other options in your README.
- **Value Added:**
Detail exactly what capabilities, simplifications, and features the framework provides to your implementation.

Evaluation criteria

Core Evaluation Criteria

- **Problem & Motivation:** Clear description and project goals.
- **Data Ingestion:** Knowledge base preparation and ingestion pipeline.
- **Retrieval Quality:** Retrieval setup and evaluation metrics.
- **RAG Flow:** End-to-end integration and LLM answering layer.
- **System Evaluation:** Comprehensive end-to-end evaluation.
- **Reproducibility:** Can a reviewer run it directly from your README?
- **Documentation:** Code structure, comments, and project clarity.

Rubric & Bonus

Each course defines its own rubric. The platform shows the exact criteria during peer reviews.

Bonus Opportunities:

- Interactive UI or web interface
- Monitoring & telemetry setups
- Cloud deployment infrastructure
- Advanced optimization techniques

Portfolio focus

Portfolio Piece: Treat the project as a professional asset. Use a meaningful repository name such as **recipes-rag-assistant** (not **homework-final**) and include a clear, professional **README** for job applications.

Start from Day One

Do not wait for the project window to open. Read the guidelines at the start, decide what to build, and apply each module as you learn. Putting concepts into immediate practice beats passive learning.

Build More Than One

Stand out from the crowd. Since everyone finishes with the same course dataset RAG app, build several projects on your own data and problems. A varied portfolio of an app, agent, or domain tool is far stronger.

Make Them Your Own

The strongest projects solve problems you actually face, using data you personally picked, cleaned, and structured. Real-world end-to-end data preparation and ownership are what truly showcase your skills to employers.